

LAB IMPLEMENTATION 5

Solidity Program Structure and Smart Contracts

BC

Aim

To study the structure and basic constructs of Solidity programs and gain hands-on experience with smart contract compilation and deployment using Remix IDE. The experiment covers the Solidity Basics course assignments and interaction with the Remix VM environment.

Theory

Solidity is a programming language used to write smart contracts for blockchain platforms. A Solidity program commonly contains a license identifier, a compiler version declaration, a contract definition, state variables, and functions. The supplied LearnETH tutorial examples demonstrate primitive data types, variables, functions, visibility, modifiers, constructors, control flow, arrays, mappings, structures, enums, data locations, and transaction-related concepts.

Functions may be declared as `pure` when they do not read or modify blockchain state, or as `view` when they read state without modifying it. Solidity also distinguishes data locations such as storage and memory. Transactions involving Ether use units such as `wei`, while gas and gas price determine the computational cost of transaction execution. Remix IDE provides an environment for compiling contracts and deploying them on the Remix VM.

Steps

1. Open the LearnETH Solidity tutorials in Remix IDE.
2. Explore the Solidity Basics course and its sequence of assignments.
3. Study primitive data types, variables, and function declarations.
4. Practice function inputs and outputs, including `pure` and `view` functions.
5. Study visibility, modifiers, constructors, and control-flow statements.
6. Practice arrays, mappings, structs, and enums as Solidity data structures.
7. Examine data locations and the use of storage and memory.
8. Study Ether and `wei`, gas, gas price, and sending transactions.
9. Compile the tutorial contracts in Remix IDE and inspect the assignment states.
10. Deploy and interact with the contracts using the Remix VM environment.

Output

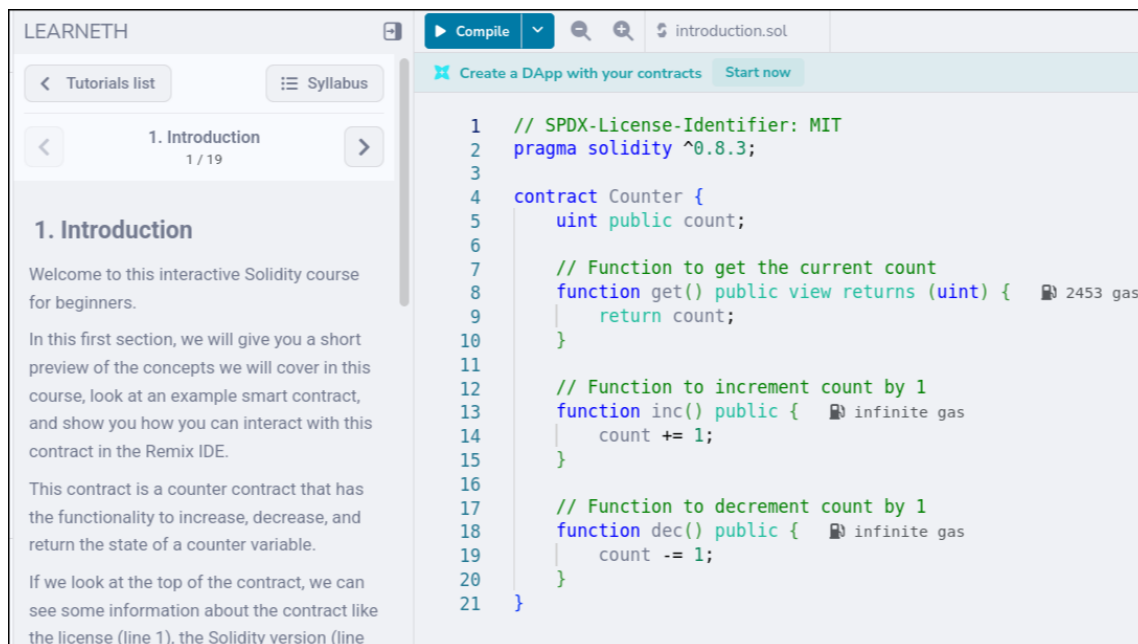


Fig. 1. LearnETH introduction with a basic Counter smart contract

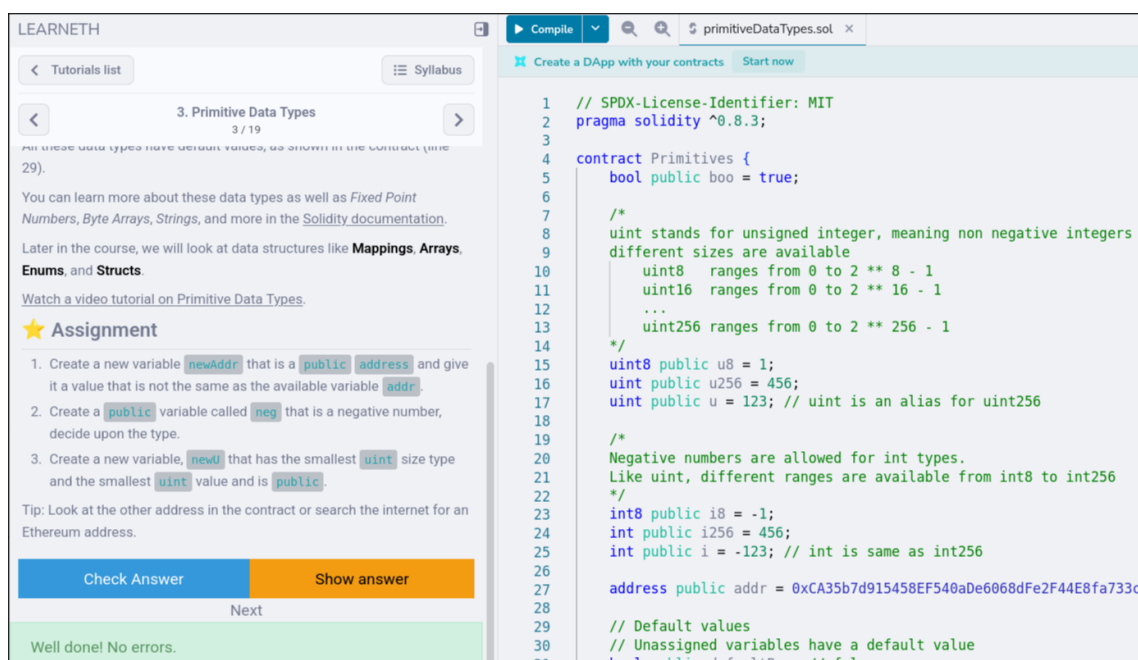


Fig. 2. Basic syntax and primitive data type assignments

LEARNETH

Tutorials list

2. Basic Syntax
2 / 19

(line 6).

Solidity is a *statically typed* language, which means that you need to specify the type of the variable when you declare it. In this case, `greet` is a `string`.

We also define the *visibility* of the variable, which specifies from where you can access it. In this case, it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Fig. 3. Basic syntax and primitive data type assignments

LEARNETH

Tutorials list

4. Variables
4 / 19

Global Variables, also called *Special Variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

Fig. 4. Variables and state variable function examples

LEARNETH

5.1 Functions - Reading and Writing to a State Variable
5 / 19

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

★ **Assignment**

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public { 22536 gas
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) { 2475 gas
16        return num;
17    }
18
19    function get_b() public view returns (bool) { 2539 gas
20        return b;
21    }
22 }
```

Fig. 5. Variables and state variable function examples

LEARNETH

5.2 Functions - View and Pure
6 / 19

b. Using inline assembly that contains certain opcodes.

From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ **Assignment**

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public { infinite gas
18        x = x + y;
19    }
20 }
```

Fig. 6. View and pure functions with modifiers and constructors

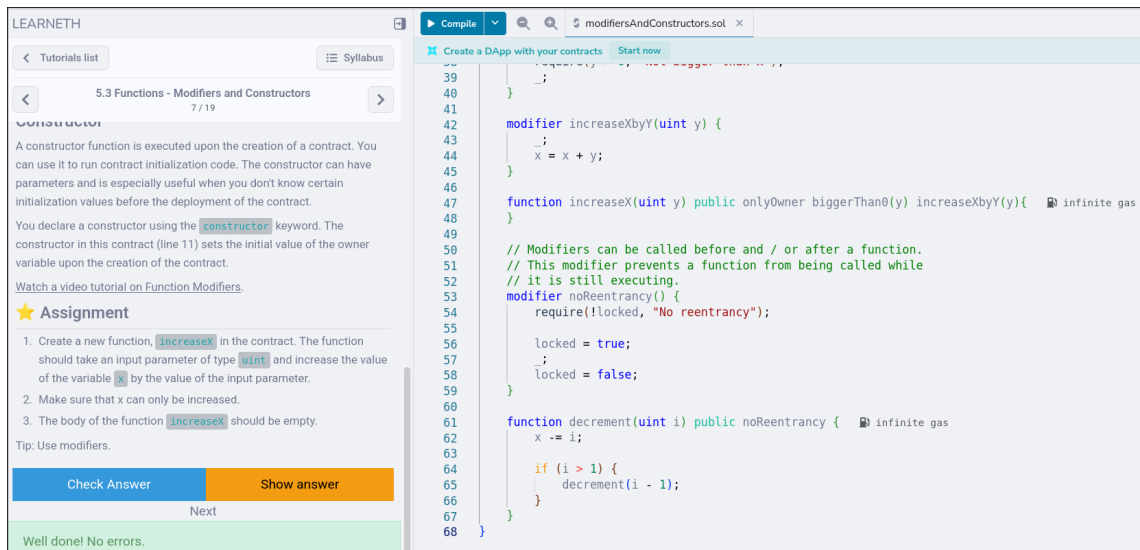


Fig. 7. View and pure functions with modifiers and constructors

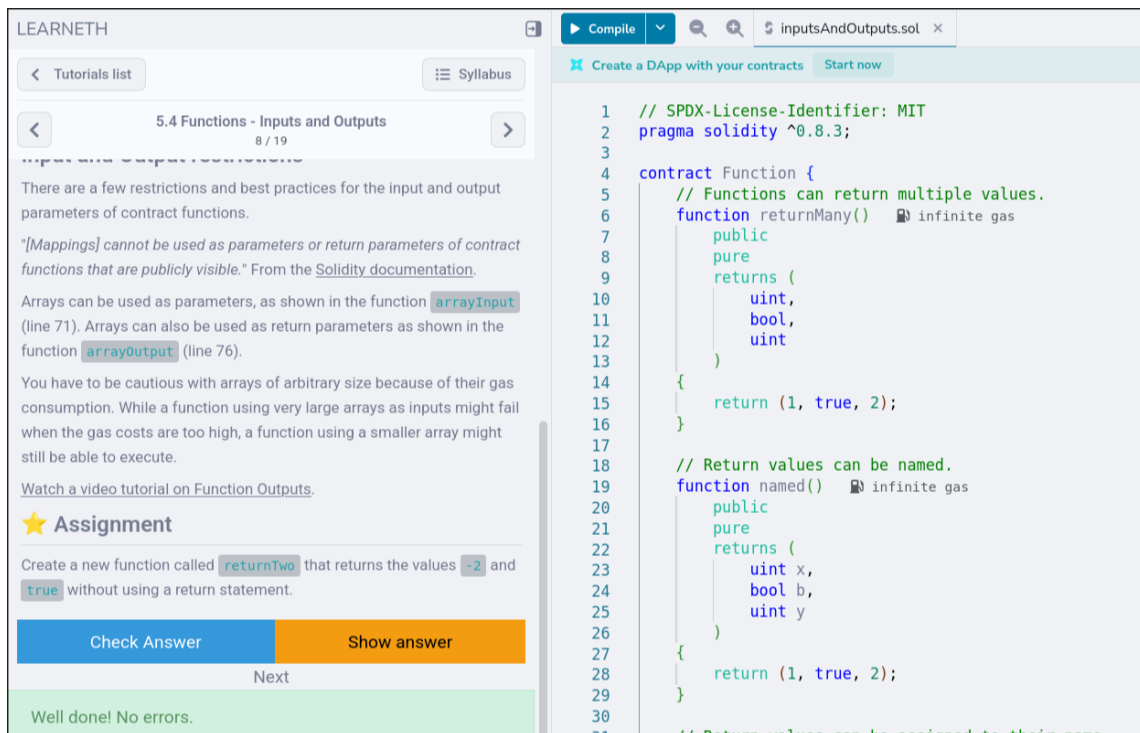


Fig. 8. Function inputs, outputs, and visibility examples

LEARNETH

6. Visibility
9 / 19

State variables can not be `external`.

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

★ **Assignment**

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

visibility.sol

```

41 // This function will not compile since we're trying to call
42 // an external function here.
43 function testExternalFunc() public pure returns (string memory) {
44     return externalFunc();
45 }
46
47 // State variables
48 string private privateVar = "my private variable";
49 string internal internalVar = "my internal variable";
50 string public publicVar = "my public variable";
51 // State variables cannot be external so this code won't compile.
52 // string external externalVar = "my external variable";
53
54
55 contract Child is Base {
56     function testInternalFunc() public pure override returns (string memory) {
57         return internalFunc();
58     }
59
60     function testInternalVar() public view returns (string memory, string memory) {
61         return (internalVar, publicVar);
62     }
63 }

```

Chaitanya

Fig. 9. Function inputs, outputs, and visibility examples

LEARNETH

7.1 Control Flow - If/Else
10 / 19

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

★ **Assignment**

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

IfElse.sol

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint _x) public pure returns (uint) {
16        // if (_x < 10) {
17        //     return 1;
18        // }
19        // return 2;
20
21        // shorthand way to write if / else statement
22        return _x < 10 ? 1 : 2;
23    }
24
25    function evenCheck(uint y) public pure returns (bool) {
26        return y%2 == 0 ? true : false;
27    }
28 }

```

Fig. 10. If-else and loop control flow assignments

The screenshot shows the LEARNETH interface. On the left, the 'Tutorials list' shows '7.2 Control Flow - Loops' (11 / 19). The main content area explains the `continue` statement, followed by the `break` statement. Below this is an 'Assignment' section with three tasks: 1. Create a public `uint` state variable called `count` in the `Loop` contract. 2. At the end of the for loop, increment the count variable by 1. 3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement. Buttons for 'Check Answer' and 'Show answer' are present, along with a 'Next' button. A green bar at the bottom says 'Well done! No errors.'

On the right, the Solidity code editor shows a file named `loops.sol`. The code defines a `contract Loop` with a public `uint count` variable. It includes a `loop()` function with a `for` loop that increments `count` and a `while` loop that increments `j`. Comments indicate the use of `continue` and `break` statements.

Fig. 11. If-else and loop control flow assignments

The screenshot shows the LEARNETH interface. On the left, the 'Tutorials list' shows '8.1 Data Structures - Arrays' (12 / 19). The main content area explains the `delete` operator and the `Array length` property. Below this is an 'Assignment' section with two tasks: 1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible. 2. Change the `getArr()` function to return the value of `arr3`. Buttons for 'Check Answer' and 'Show answer' are present, along with a 'Next' button. A green bar at the bottom says 'Well done! No errors.'

On the right, the Solidity code editor shows a file named `arrays.sol`. The code defines a `contract Arrays` with a public `uint[] arr` array. It includes a `remove` function that removes an element from the array and a `test` function that pushes elements and removes them. Comments indicate the use of `delete` and `pop` methods.

Fig. 12. Arrays and mappings data structure assignments

The screenshot shows the LEARNETH interface with a tutorial on Mappings (8.2 Data Structures - Mappings) on the left and a Solidity code editor on the right. The tutorial includes an assignment with three steps: creating a public mapping, changing functions to work with the mapping, and creating a new entry. The code editor shows a Solidity contract named 'NestedMapping' with functions for deleting, getting, and setting values in a nested mapping.

LEARNETH

8.2 Data Structures - Mappings

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer Show answer

Next

Well done! No errors.

```
20 // Reset the value to the default value.
21 delete balances[_addr];
22 }
23 }
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;
28
29     function get(address _addr1, uint _i) public view returns (bool) {
30         // You can get values from a nested mapping
31         // even when it is not initialized
32         return nested[_addr1][_i];
33     }
34
35     function set(
36         address _addr1,
37         uint _i,
38         bool _boo
39     ) public {
40         nested[_addr1][_i] = _boo;
41     }
42
43     function remove(address _addr1, uint _i) public {
44         delete nested[_addr1][_i];
45     }
46 }
```

Fig. 13. Arrays and mappings data structure assignments

The screenshot shows the LEARNETH interface with a tutorial on Structs (8.3 Data Structures - Structs) on the left and a Solidity code editor on the right. The tutorial includes an assignment with one step: creating a function to remove a struct member. The code editor shows a Solidity contract named 'Todos' with a struct 'Todo' and an array 'todos'.

LEARNETH

8.3 Data Structures - Structs

General parameters: We can provide the name of the struct and the values of its members as parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ **Assignment**

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Todos {
5     struct Todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function create(string memory _text) public {
14        // 3 ways to initialize a struct
15        // - calling it like a function
16        todos.push(Todo(_text, false));
17
18        // key value mapping
19        todos.push(Todo({text: _text, completed: false}));
20
21        // initialize an empty struct and then update it
22        Todo memory todo;
23        todo.text = _text;
24        // todo.completed initialized to false
25
26        todos.push(todo);
27    }
28
29    // Solidity automatically created a getter for 'todos' so
30    // you don't actually need this function.
31    function get(uint _index) public view returns (string memory text, bool completed) {
32        return (todos[_index].text, todos[_index].completed);
33    }
34 }
```

Fig. 14. Structs and enums data structure assignments

LEARNETH

Tutorials list | Syllabus

8.4 Data Structures - Enums
15 / 19

name of the variable that is storing the value (line 25).

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer | Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    enum Size {
15        S,
16        M,
17        L
18    }
19
20    // Default value is the first element listed in
21    // definition of the type, in this case "Pending"
22    Status public status;
23    Size public sizes;
24
25    function get() public view returns (Status) { 2605 gas
26        return status;
27    }
28
29    function getSize() public view returns (Size) { 2633 gas
30        return sizes;
31    }

```

Fig. 15. Structs and enums data structure assignments

LEARNETH

Tutorials list | Syllabus

9. Data Locations
16 / 19

in the mapping `myStructs` (line 10).

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer | Show answer

Next

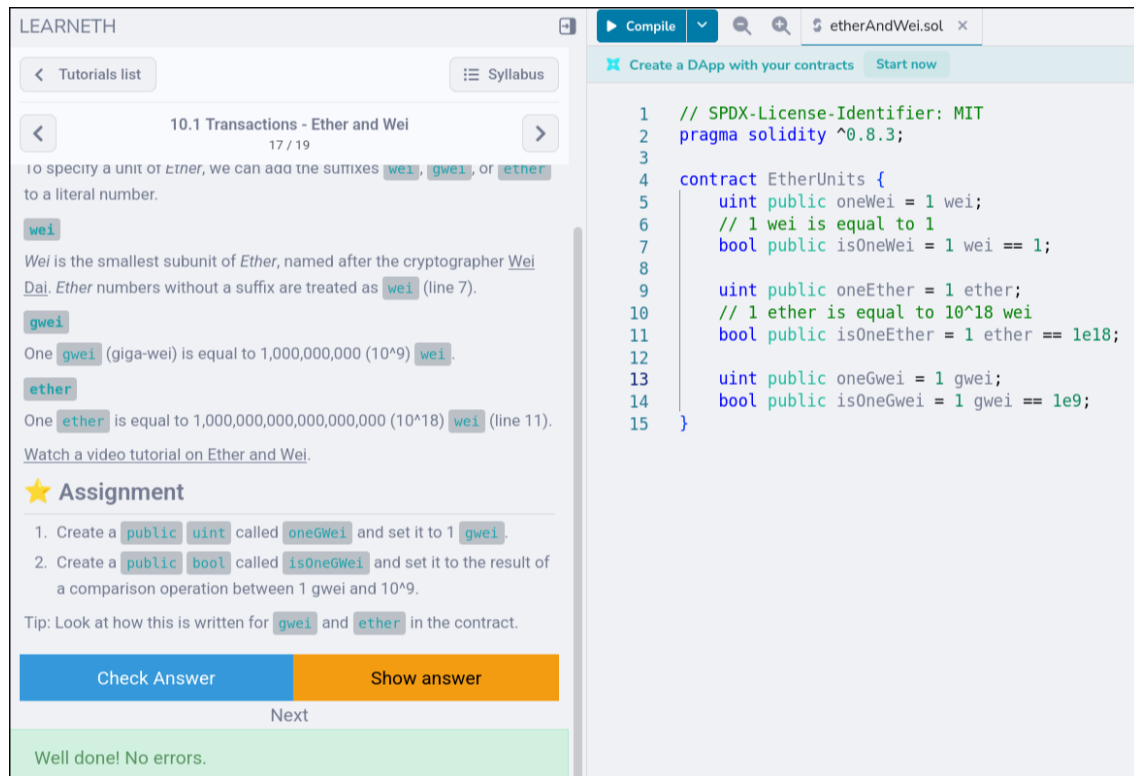
Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) public myStructs;
11
12    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
13        // call _f with state variables
14        _f(arr, map, myStructs[1]);
15        // get a struct from a mapping
16        MyStruct storage myStruct = myStructs[1];
17        myStruct.foo = 4;
18        // create a struct in memory
19        MyStruct memory myMemStruct = MyStruct(0);
20        MyStruct memory myMemStruct2 = myMemStruct;
21        myMemStruct2.foo = 1;
22
23        MyStruct memory myMemStruct3 = myStruct;
24        myMemStruct3.foo = 3;
25        return (myStruct, myMemStruct2, myMemStruct3);
26    }
27
28    function _f( undefined gas
29        uint[] storage _arr,
30        mapping(uint => address) storage _map,
31        MyStruct storage myStruct

```

Fig. 16. Data locations and Ether and wei examples



LEARNETH

Tutorials list | Syllabus

10.1 Transactions - Ether and Wei

17 / 19

To specify a unit or *Ether*, we can add the suffixes `wei`, `gwei`, or `ether` to a literal number.

`wei`

Wei is the smallest subunit of *Ether*, named after the cryptographer *Wei Dai*. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on [Ether and Wei](#).

★ Assignment

- Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
- Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

[Check Answer](#) [Show answer](#)

Next

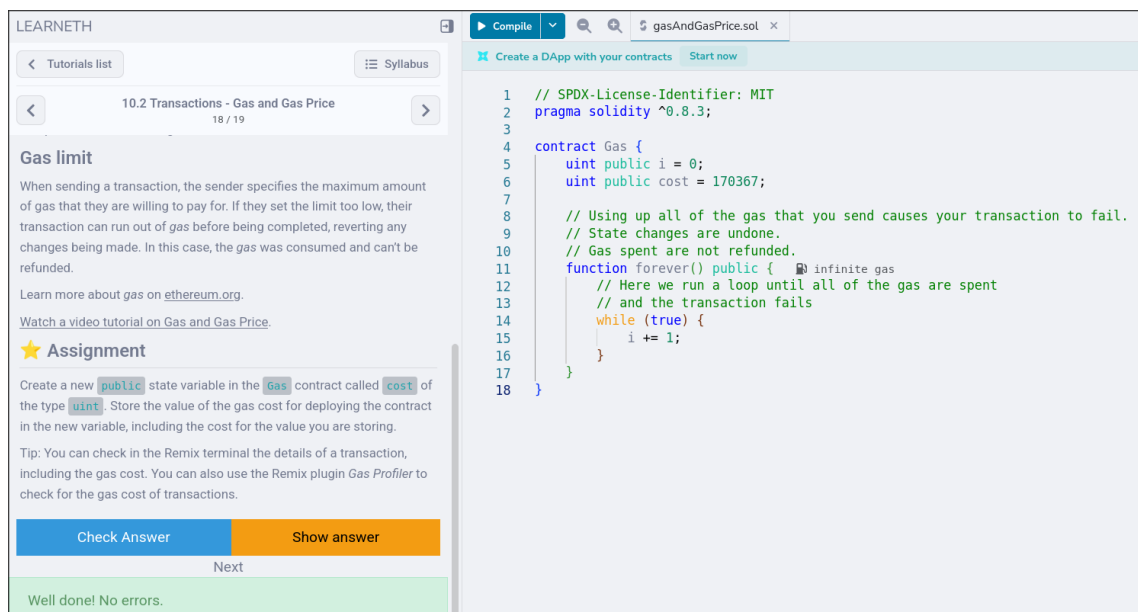
Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    bool public isOneGwei = 1 gwei == 1e9;
15 }

```

Fig. 17. Data locations and Ether and wei examples



LEARNETH

Tutorials list | Syllabus

10.2 Transactions - Gas and Gas Price

18 / 19

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about *gas* on [ethereum.org](#).

Watch a video tutorial on [Gas and Gas Price](#).

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }

```

Fig. 18. Gas price and sending Ether transaction examples

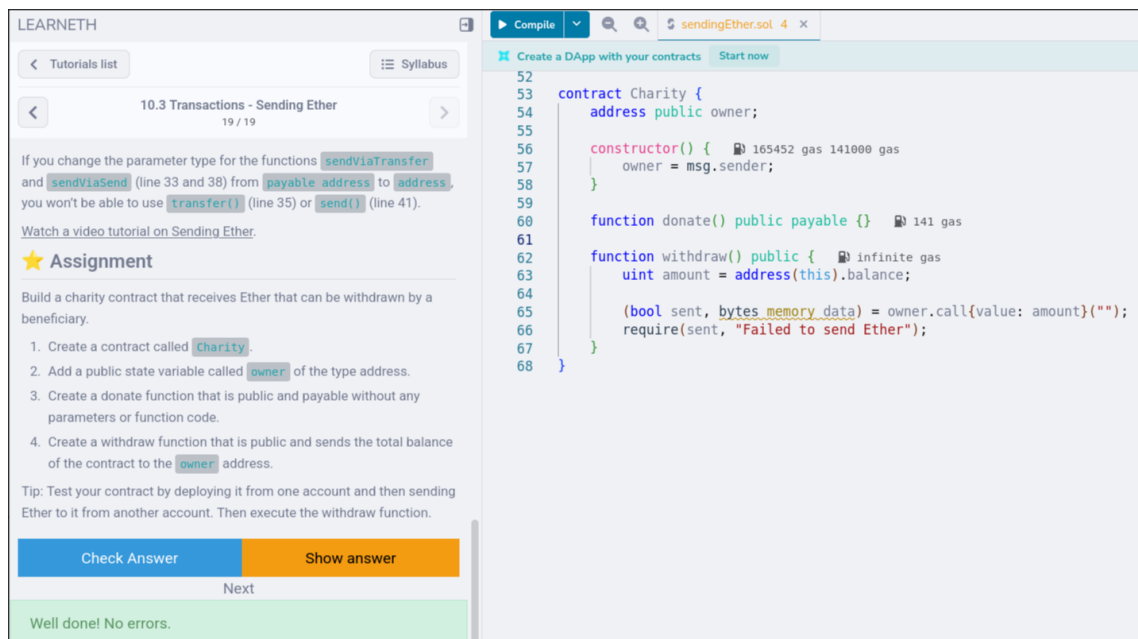


Fig. 19. Gas price and sending Ether transaction examples

Conclusion

The Solidity Basics course in Remix IDE was studied using the supplied tutorial material. The examples covered smart contract structure, primitive data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, data locations, and transaction concepts. The supplied screenshots showed the tutorial contracts and completed assignment states in the Remix environment. The experiment provided hands-on exposure to Solidity programming and smart contract deployment concepts using Remix VM.